

AegisAI — Services, Contracts & Schema

Project: AegisAI — Enterprise AI Security & Governance Gateway

Submission covers: Task 1 (Capability List), Services and Ownership, Task 3 (Service Contracts), Task 4 (Full Specification of the Central Operation), Task 6 (Service Property Validation)

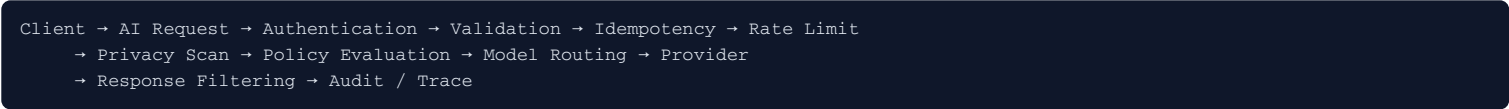
Task 1 — Capability List	Page 2
Services and Ownership	Page 5
Task 3 — Service Contracts	Page 6
Task 4 — Full Specification of <code>submitAIRequest</code>	Page 8
Task 6 — Service Property Validation	Page 10

Task 1: Capability List

1. Purpose

AegisAI is a security and governance gateway placed between enterprise applications or AI agents and AI model providers. The system receives an AI request, authenticates the caller, validates the request, checks rate limits and idempotency, scans for sensitive information, evaluates organizational policies, selects an appropriate AI model, calls the provider with reliability protection, and records the request and security trace.

The main flow is:



The system has four main user / actor types:

- Organization Administrator
- Developer
- Security Auditor
- AI Agent / Service Account

2. Method

Each Capability was derived by pairing every actor named in the AegisAI project blueprint (Organization Administrator, Developer, Security Auditor, AI Agent / Service Account) with every module and data entity the blueprint describes. Capabilities the blueprint names directly — submitting an AI request, evaluating a policy, scanning for sensitive data, routing to a model, recording a trace — are marked as **directly stated**. Capabilities that are necessary for the system to function but are only implied by the blueprint's module descriptions, most notably identity/access management (C1–C5) and request idempotency (C9), are marked as **inferred** and are justified individually in Section 3. For every Capability, the responsible actor, the Data Entities it reads or writes, and a formal description are recorded, so that each Capability can later be traced to exactly one owning service, since no two services may share data.

3. Assumptions Behind the Inferred Capabilities

The blueprint states that AegisAI serves four actor types and that access to functionality depends on organizational role and on the permissions attached to a service account. A multi-tenant system with role-dependent behaviour cannot function without a way to register an organization, create accounts within it, prove identity before each request, and issue machine credentials for AI agents. Capabilities C1 through C5 (Identity and Access Management) are therefore treated as first-class Capabilities on the same footing as the capabilities named directly in the blueprint's request-processing flow, because Identity and Access typically forms its own bounded context once services are drawn, and omitting it at this stage would leave a gap the service design depends on. Likewise, C9 (Enforce Request Idempotency) is inferred from the blueprint's documented requirement for an `Idempotency-Key` header on `POST /v1/ai-requests` (checklist item 2), and C10 (Enforce Rate Limits) is inferred from the documented rate-limit headers and `429` response (checklist item 12), rather than being named as standalone capability verbs in the blueprint's prose.

4. Capability Catalogue

A. Identity and Access Management

ID	Capability	Actor	Data Entities	Description
C1	Register an Organization	Organization Administrator	Organization	Creates an organization and stores its basic account and plan information.
C2	Authenticate a User	Organization Administrator / Developer / Security Auditor	User	Verifies user credentials and creates an authenticated session using JWT.
C3	Manage Users	Organization Administrator	User	Creates, updates, disables, and manages users belonging to an organization.
C4	Manage Service Accounts	Organization Administrator	Service Account	Creates machine-to-machine identities for AI agents and applications with defined permissions.
C5	Verify a Session	System / Services	User	Verifies a JWT/session and returns the caller identity, organization, and role.

B. AI Request Management

ID	Capability	Actor	Data Entities	Description
C6	Submit an AI Request	Developer / AI Agent / Service Account	AI Request	Accepts an AI request and sends it through the complete AegisAI security and governance pipeline.
C7	Retrieve an AI Request	Developer / Security Auditor	AI Request	Returns the status, selected model, result information and trace reference of a request.
C8	List AI Requests	Developer / Security Auditor / Organization Administrator	AI Request	Returns paginated AI request history with filters.
C9	Enforce Request Idempotency	System	AI Request / Redis State	Prevents duplicate processing when the same POST request is submitted with the same idempotency key.

C10	Enforce Rate Limits	System	Redis Rate-Limit State	Restricts request frequency according to configured organizational/user limits.
-----	---------------------	--------	------------------------	---

C. Privacy Scanning

ID	Capability	Actor	Data Entities	Description
C11	Scan Sensitive Data	System	Scanner Result	Detects PII, credentials, API keys, secrets and configurable sensitive patterns.

This capability is owned by a dedicated **Privacy Scanner Service** so detection/masking logic is isolated from policy decisions and from audit storage.

D. Policy Governance

ID	Capability	Actor	Data Entities	Description
C12	Evaluate an AI Policy	Organization Administrator	Policy	Determines whether an AI request should be allowed, blocked, masked, routed, or sent for approval according to the organization's policies.
C13	Manage Policies	Organization Administrator	Policy	Creates, updates, activates and deactivates organization policies.
C14	Approve or Reject a Request	Authorized Organization Administrator	Approval	Handles requests that require human approval before processing.

AegisAI's documented policy actions are **ALLOW, BLOCK, MASK, REQUIRE_APPROVAL** and **ROUTE**.

C12's actor is recorded as Organization Administrator rather than System: the evaluation itself runs automatically inside the Policy Service, but the policy being evaluated belongs to the organization and encodes its governance decisions, so the organization is the business owner of the capability even though the system is the executor.

E. Model and Provider Management

ID	Capability	Actor	Data Entities	Description
C15	Register an AI Provider	Organization Administrator	Provider	Registers a provider such as OpenAI, Gemini or a controlled local provider.
C16	Register a Model	Organization Administrator	Model	Adds a model with its provider, capabilities, sensitivity level and cost information.
C17	Select a Model	System	Model / Provider	Selects a suitable model according to policy, capability, cost and provider health.
C18	Invoke an AI Provider	System	Provider / Model	Sends the approved request to the selected provider using timeout, retry and circuit-breaker protection.

Model routing is based on factors such as request complexity, sensitive data, provider health and organization budget.

F. Security, Audit and Observability

ID	Capability	Actor	Data Entities	Description
C19	Record an AI Request Trace	System	Request Trace	Records the sequence of important events belonging to an AI request.
C20	Record a Security Event	System	Security Event	Records PII detection, secret detection, policy violations and other security events.
C21	View Audit Logs	Security Auditor / Organization Administrator	Audit Event	Retrieves auditable system and security activity.
C22	View Request Trace	Developer / Security Auditor	Request Trace	Displays the lifecycle of a request using its correlation/trace ID.
C23	Monitor Provider Health	Organization Administrator / System	Provider Health	Tracks provider failures, circuit state, availability, and health information.

G. Enterprise Integration

ID	Capability	Actor	Data Entities	Description
C24	Invoke a Legacy SOAP Service	System	SOAP Integration State	Reads the WSDL and performs a controlled SOAP call to a legacy enterprise system.
C25	Handle SOAP Failure	System	Integration Trace	Applies timeout, retry/circuit-breaker protection, and failure handling when the legacy SOAP dependency is unavailable.

The SOAP/WSDL connector is included to demonstrate legacy enterprise interoperability, as required by the course checklist.

5. Traceability to the Blueprint

Capabilities C1 through C5 are inferred from the multi-tenant, role-based access model described throughout the blueprint, as justified in Section 3. Capabilities C6 through C10 correspond directly to the AI Gateway's documented request-processing flow (submit, retrieve, list, idempotency, rate limiting). Capability C11 corresponds to the Privacy Scanner module the blueprint names explicitly as a distinct responsibility. Capabilities C12 through C14 correspond to the Policy Engine's documented `ALLOW` / `BLOCK` / `MASK` / `ROUTE` /

`REQUIRE_APPROVAL` actions. Capabilities C15 through C18 correspond to the Model Router and Provider Adapter modules. Capabilities C19 through C23 correspond to the Audit & Observability module and its documented correlation-ID and trace requirements (checklist item 26). Capabilities C24 and C25 correspond to the Legacy SOAP Connector the blueprint requires as a course checklist item (item 24).

Services and Ownership

The twenty-five Capabilities from Task 1 are grouped into six services, each formed around the Data Entity (or small cluster of entities) its capabilities touch. This is the benchmark diagram the team built to, together with the member responsible for implementing each service.

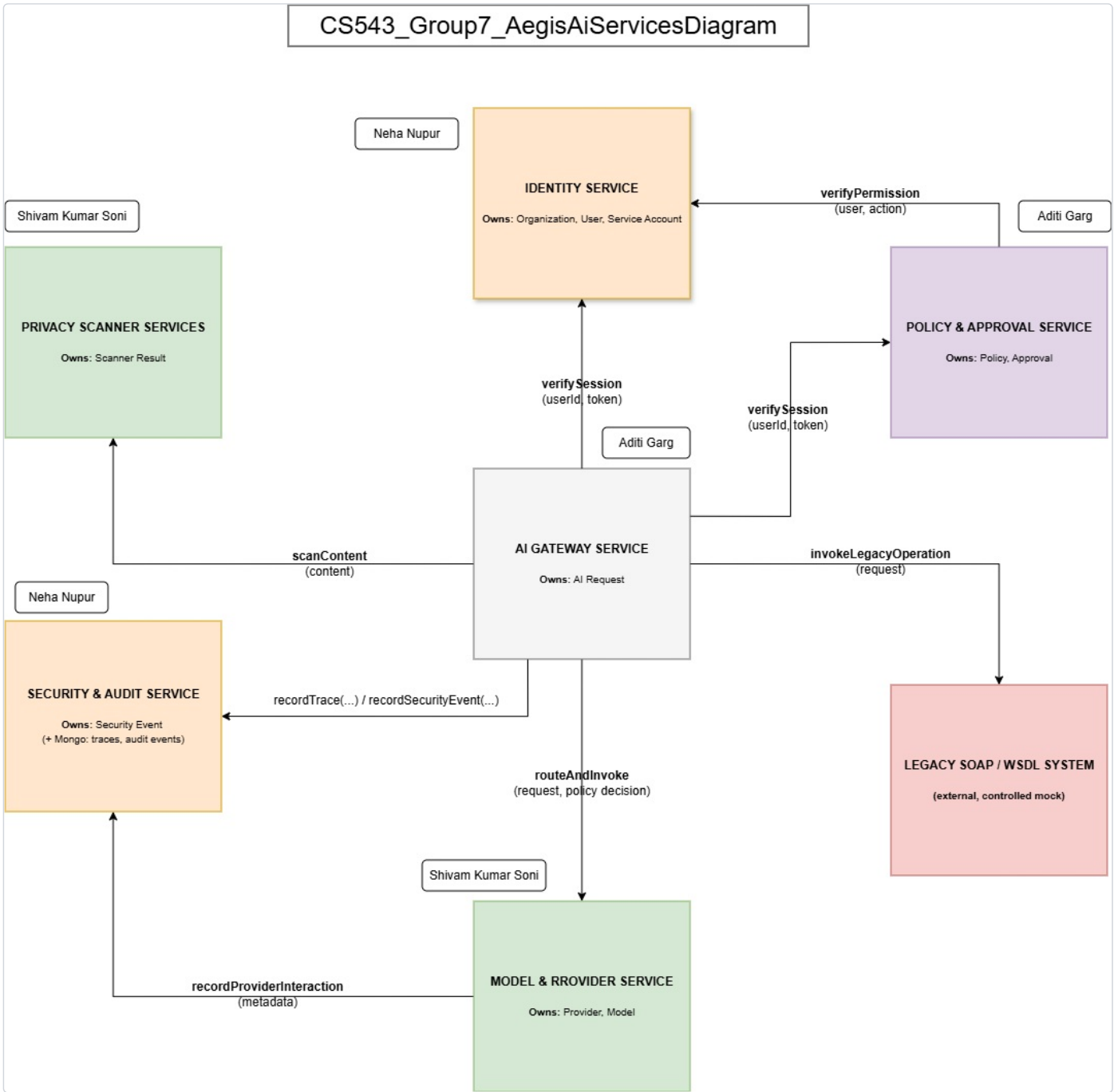


Figure 1. AegisAI service boundaries, cross-service calls, and the team member responsible for each service (CS543_Group7_AegisAIServicesDiagram).

Service	Owns (Data)	Team Member	Capabilities It Implements (from Task 1)
Identity Service	Organization, User, Service Account	Neha Nupur	C1 Register an Organization, C2 Authenticate a User, C3 Manage Users, C4 Manage Service Accounts, C5 Verify a Session
Privacy Scanner Service	Scanner Result	Shivam Kumar Soni	C11 Scan Sensitive Data
AI Gateway Service	AI Request	Aditi Garg	C6 Submit an AI Request, C7 Retrieve an AI Request, C8 List AI Requests, C9 Enforce Request Idempotency, C10 Enforce Rate Limits, and orchestrates C24-C25 (Legacy SOAP Connector)
Policy & Approval Service	Policy, Approval	Aditi Garg	C12 Evaluate an AI Policy, C13 Manage Policies, C14 Approve or Reject a Request
Model & Provider Service	Provider, Model	Shivam Kumar Soni	C15 Register an AI Provider, C16 Register a Model, C17 Select a Model, C18 Invoke an AI Provider
Security & Audit Service	Security Event, Request Trace, Audit Event	Neha Nupur	C19 Record an AI Request Trace, C20 Record a Security Event, C21 View Audit Logs, C22 View Request Trace, C23 Monitor Provider Health

Task 3: Service Contracts

1. Purpose

This section defines the contract exposed by every service identified above. A contract lists only what a service promises to the outside world — the operation name, the input it needs, the output it returns, and the errors it can report. It never mentions how the service stores its data, what its tables look like, or what the internal identifiers are shaped like. The inside of each service stays hidden, exactly as the service boundaries require.

2. Contract Principle

A service contract exposes only: **operation name, input, output, and errors.**

It must **not** expose: database tables, SQL, internal algorithms, internal identifiers, Redis keys, or any other internal implementation details.

This follows the same contract principle used in the CampusEats benchmark: a service requiring another service's data must call that service's contract rather than reading its storage directly.

AI Gateway Service and Policy & Approval Service are treated as the two **main services**, since together they carry the core request-processing journey — accepting, validating, scanning, and deciding the fate of an AI request before it ever reaches a model — so each gets three or more operations below. The remaining four services get at least one operation each, as the task requires.

3.1 AI Gateway Service — Main Service

The AI Gateway is the **primary service** because the central purpose of AegisAI is to control and process AI requests.

Operation	Input	Output	Errors
submitAIRequest	Session token, organization context, prompt/messages, optional model preference, idempotency key	Request reference, status, selected model, response/result or processing state, trace ID	NotAuthenticated, InvalidRequest, RateLimitExceeded, PolicyDenied, SensitiveDataBlocked, ApprovalRequired, NoModelAvailable, ProviderUnavailable
getAIRequest	Request reference + caller identity	Request status, selected model, result metadata, trace ID	RequestNotFound, NotAuthorised
listAIRequests	Caller identity, optional status/filter, cursor	Paginated request list + next cursor/link	NotAuthorised, InvalidCursor
getRequestTrace	Request reference + caller identity	Trace reference/status	RequestNotFound, NotAuthorised

The REST blueprint defines `POST /v1/ai-requests`, `GET /v1/ai-requests/:id`, and `GET /v1/ai-requests`, with the POST operation returning `202 Accepted`.

3.2 Privacy Scanner Service

Operation	Input	Output	Errors
scanContent	Request content, organization context	Detected categories, actions, and sanitized/masked content	ScannerError, InvalidContent
maskSensitiveData	Content, detected findings	Masked content	ScannerError
getScanResult	Scan reference + caller identity	Scan result details (categories, actions, status)	ScanResultNotFound, NotAuthorised

Kept as its own service boundary so detection/masking rules can evolve (deterministic patterns today, a specialized detector later) without touching policy logic or audit storage. Detects email addresses, phone numbers, PAN-like patterns, Aadhaar-like patterns, API keys, secrets, password/credential patterns, and organization-sensitive keywords.

3.3 Policy & Approval Service — Main Service

Operation	Input	Output	Errors
createPolicy	Organization identity, policy name, rules, action configuration	Policy reference and version	NotAuthorised, ValidationError, DuplicatePolicy
updatePolicy	Policy reference, organization identity, changed rules	Updated policy version	PolicyNotFound, NotAuthorised, ValidationError
evaluatePolicy	Organization, caller, request metadata, model information, scanner result	ALLOW, BLOCK, MASK, ROUTE or REQUIRE_APPROVAL decision	PolicyNotFound, EvaluationError
getPolicy	Policy reference + caller identity	Policy details and current version	PolicyNotFound, NotAuthorised
approveRequest	Approval reference, approver identity, reason	Approval status	ApprovalNotFound, NotAuthorised, AlreadyResolved
rejectRequest	Approval reference, approver identity, reason	Rejection status	ApprovalNotFound, NotAuthorised, AlreadyResolved

3.4 Identity Service

Operation	Input	Output	Errors
authenticate	Email/credentials	JWT/session token, user reference, role	InvalidCredentials, AccountDisabled

verifySession	Session token	Caller identity, organization, role, token validity	InvalidSession, SessionExpired
createUser	Organization identity, user information, role	User reference	DuplicateEmail, ValidationError
createServiceAccount	Organization identity, name, permissions	Service-account reference and credentials metadata	NotAuthorised, ValidationError
getUser	User reference + caller identity	User profile	UserNotFound, NotAuthorised

3.5 Model & Provider Service

Operation	Input	Output	Errors
listModels	Organization identity, optional capability/filter	Available models and providers	NotAuthorised
registerProvider	Provider configuration	Provider reference	ValidationError, NotAuthorised
registerModel	Provider reference, model information	Model reference	ProviderNotFound, ValidationError
routeAndInvoke	AI request, policy decision, model preference	Provider response, model used, latency/status	NoModelAvailable, ProviderUnavailable, Timeout, CircuitOpen
getProviderHealth	Provider reference	Health and circuit state	ProviderNotFound

Providers/models are structured PostgreSQL data, and the contract explicitly surfaces provider status and circuit state without exposing how they are computed internally.

3.6 Security & Audit Service

Operation	Input	Output	Errors
recordSecurityEvent	Organization, event type, severity, request/trace reference	Security-event reference	ValidationError
recordTrace	Request reference, trace ID, ordered events	Trace reference	ValidationError, StorageError
getTrace	Trace ID + caller identity	Ordered request trace	TraceNotFound, NotAuthorised
listSecurityEvents	Organization, filters, cursor	Paginated security events	NotAuthorised, InvalidCursor
getAuditLogs	Organization, filters, cursor	Paginated audit events	NotAuthorised, InvalidCursor

MongoDB holds request traces, provider interactions and audit events behind this service's contract. Scanner results now live behind the Privacy Scanner Service's own contract instead.

4. Cross-Service Calls Implied by the Contracts

No two services share a table, so any operation that needs data owned by another service must call that service's contract instead of reaching into its storage. The contracts above create the following calling pattern, matching Figure 1, and carried forward into the internal walk-through of `submitAIRequest` in Task 4.

Caller	Calls	Reason
AI Gateway Service	Identity Service - <code>verifySession</code>	To confirm who is making the AI request and which organization they belong to
AI Gateway Service	Privacy Scanner Service - <code>scanContent</code>	To detect PII, secrets, and other sensitive content before policy evaluation
AI Gateway Service	Policy & Approval Service - <code>evaluatePolicy</code>	To decide whether the request should be allowed, blocked, masked, routed, or held for approval
AI Gateway Service	Model & Provider Service - <code>routeAndInvoke</code>	To select a model/provider and obtain the AI response
AI Gateway Service	Security & Audit Service - <code>recordTrace</code> / <code>recordSecurityEvent</code>	To persist the request's lifecycle and any security findings
AI Gateway Service	Legacy SOAP/WSDL System - <code>invokeLegacyOperation</code>	To demonstrate the required legacy enterprise interoperability path (checklist item 24)
Policy & Approval Service	Identity Service - <code>verifyPermission</code>	To confirm the caller is authorised to create, update, or approve a policy
Model & Provider Service	Security & Audit Service - <code>recordProviderInteraction</code>	To log provider latency/status without exposing it through AI Gateway Service

Task 4: Full Specification of submitAIRequest

1. Why submitAIRequest Is the Central Operation

submitAIRequest is the central operation of AegisAI because almost every important capability exists either to prepare, protect, process or observe an AI request. This is the exact core demo flow defined in the AegisAI project blueprint:

```
Authentication → Validation → Idempotency → Rate Limit → Privacy Scan
→ Policy → Model Routing → Provider → Response Filtering → Audit/Trace
```

2. Public Contract

2.1 Input

Field	Meaning
Session token	Proves that the caller is authenticated.
Organization reference	Identifies the organization making the request; normally derived from the authenticated identity.
Messages / Prompt	The AI content that should be processed.
Model preference	Optional requested model or model class.
Idempotency key	Prevents accidental duplicate processing of the same POST request.
Request metadata	Optional application/request information required for tracing and policy evaluation.

The exact payload fields are not completely frozen in the current AegisAI blueprint, so these fields are treated as the **Assignment 2 design contract** rather than an already-implemented schema.

2.2 Output on Success

Field	Meaning
Request reference	Unique reference for the AI request.
Status	Current request state, such as accepted, processing, completed or blocked.
Model used	Model/provider selected by AegisAI.
Response	Controlled model response when processing completes synchronously.
Trace ID	Used to inspect the complete request lifecycle.
Created at	Time when AegisAI accepted the request.

For the REST API, the documented endpoint uses 202 Accepted for submitting an AI request.

2.3 Error Cases

Error	When It Is Reported
NotAuthenticated	Session token is missing, invalid or expired.
InvalidRequest	Required fields are missing or have invalid types.
RateLimitExceeded	The organization/user has exceeded its configured limit.
IdempotencyConflict	The same key is reused with a different request.
SensitiveDataBlocked	A detected sensitive value violates the configured policy.
PolicyDenied	The policy engine rejects the request.
ApprovalRequired	The request requires human approval before execution.
NoModelAvailable	No model satisfies the policy and routing requirements.
ProviderUnavailable	No usable provider is currently available.
ProviderTimeout	Provider does not respond within the configured timeout.
CircuitOpen	The provider circuit breaker is currently open.
InternalServerError	Unexpected internal failure.

3. Internal Processing — Hidden From Callers

The following implementation details remain hidden from the client; only the contract in Section 2 is public.

- Authenticate the caller.** AI Gateway sends the session token to Identity Service → verifySession() . If authentication fails, the request stops with NotAuthenticated .
- Validate request structure.** The gateway validates the request using the JSON Schema/AJV layer — required fields, data types, message structure, model preference format, idempotency key format. Invalid input returns InvalidRequest .
- Check idempotency.** The gateway checks Redis for the supplied idempotency key. If the same key and same request were already processed, AegisAI returns the earlier result rather than calling the provider again. If the key exists with a different request, it returns IdempotencyConflict . Redis is used for idempotency records and short-lived request state.

4. **Check rate limit.** The gateway checks the organization's/user's request limit. If exceeded, it returns `RateLimitExceeded` , mapped to HTTP `429 Too Many Requests` with rate-limit headers.
5. **Scan the request.** AI Gateway calls `Privacy Scanner Service → scanContent()` , which checks for email addresses, phone numbers, PAN-like patterns, Aadhaar-like patterns, API keys, secrets, password/credential patterns, and organization-sensitive keywords. Deterministic rules are used for the first version rather than claiming perfect PII detection.
6. **Evaluate policy.** AI Gateway sends the request context and scanner result to `Policy Service → evaluatePolicy()` . The result is one of `ALLOW` , `BLOCK` , `MASK` , `ROUTE` OR `REQUIRE_APPROVAL` .
7. **Apply the policy decision.** `BLOCK` stops processing and creates a security event. `MASK` replaces sensitive values and continues. `REQUIRE_APPROVAL` creates an approval record and pauses processing. `ROUTE` selects a permitted model/provider. `ALLOW` continues normally.
8. **Select the model.** AI Gateway calls `Model & Provider Service → routeAndInvoke()` , which chooses a model based on policy restrictions, sensitivity requirements, model capability, cost, provider health and availability — e.g. a sensitive request routes to an approved private/local model, a simple request routes to a lower-cost model, and an unhealthy provider triggers a fallback provider.
9. **Call the provider.** The provider adapter makes the external AI call with timeout, retry, exponential backoff, circuit breaker, and fallback where configured.
10. **Filter the response.** The provider response is treated as untrusted external data; AegisAI performs any required response filtering before returning it to the caller.
11. **Record the trace.** The system records the sequence `Request → Scanner → Policy → Router → Provider → Response` using the request's correlation/trace ID. Raw passwords, API keys and full sensitive prompts are never stored in traces.
12. **Return the result.** The gateway returns only the public response: request reference, status, model/provider, response/result, trace ID, and timestamp.

The caller never sees internal service calls, database tables, Redis keys, provider API credentials, internal retry details, scanner implementation, or SQL queries.

Task 6: Service Property Validation

1. Purpose

This task checks every one of the six AegisAI services against five properties a well-designed service is expected to have: reachable, self-contained, has a contract, independent, and loosely coupled. The check is not a formality — AI Gateway Service genuinely strains against Independent once its cross-service calls are followed through carefully, and Model & Provider Service strains against it more narrowly, and this section says so plainly rather than marking every box as a pass. Where a property does not fully hold, a concrete fix is proposed that keeps the service boundary intact.

2. What Each Property Means Here

Property	What is being checked
Reachable	The service exposes its operations through a callable interface — another service or client can find it and invoke it. It is not logic buried inside another service's process.
Self-contained	The service owns and manages all the data it needs to do its job. No other service holds a piece of its state, and it does not reach into another service's storage to do its own work.
Has a contract	The service publishes an explicit interface — operation name, input, output, and errors — and callers rely on that interface rather than on internal details. This is the Task 3 deliverable.
Independent	The service can keep doing its job — be deployed, changed, and, most importantly, stay up — without being tied to another service's availability at the moment it is called.
Loosely coupled	Wherever the service touches another service, it does so only through that service's contract, never by depending on its internal schema, its storage, or its implementation choices.

Note on Independent vs. Loosely coupled. These two are easy to conflate but check different things. A service can be loosely coupled — every cross-service call goes through a published operation, never a shared table — while still failing to be independent, if that operation has to succeed synchronously, in real time, before the calling service can finish its own job. AI Gateway Service, below, is exactly this case: every call it makes is contract-based (loosely coupled, pass), but its central operation cannot complete unless three other services are up at that moment (independent, fail).

3. Validation Table

Service	Reachable	Self-contained	Has a Contract	Independent	Loosely Coupled
Identity Service	Pass	Pass	Pass	Pass	Pass
AI Gateway Service	Pass	Pass	Pass	Fail	Pass
Privacy Scanner Service	Pass	Pass	Pass	Pass	Pass
Policy & Approval Service	Pass	Pass	Pass	Pass	Pass
Model & Provider Service	Pass	Pass	Pass	Partial	Pass
Security & Audit Service	Pass	Pass	Pass	Pass	Pass

4. Per-Service Analysis

4.1 Identity Service

- Reachable — Pass.** Exposes authentication, session verification and user/service-account operations.
- Self-contained — Pass.** Owns Organizations, Users, and Service Accounts, and never directly accesses another service's database.
- Has a contract — Pass.** Operations such as `authenticate()` and `verifySession()` are explicitly defined.
- Independent — Pass.** Identity operations work without Policy, Model or Audit Service being available.
- Loosely coupled — Pass.** Any interaction with another service occurs through a published contract.

4.2 AI Gateway Service

- Reachable — Pass.** Exposes the main REST resource `POST /v1/ai-requests` and related request operations.
- Self-contained — Pass.** Owns the `ai_requests` data and its request-specific state; does not directly read user, policy, model, or security tables.
- Has a contract — Pass.** `submitAIRequest`, `getAIRequest`, and `listAIRequests` are explicitly defined.
- Independent — Fail.** This is the main weakness. `submitAIRequest` depends on several services before it can safely process an AI request:



If these dependencies are unavailable, the central request-processing operation may not complete. This mirrors the CampusEats Order Service case, where a service can be loosely coupled while still not being independent because its central operation depends synchronously on other services.

Loosely coupled — Pass. The gateway never reads another service's database directly; all communication uses service contracts.

Fix. The dependency problem can be reduced by: caching recently verified sessions; keeping a versioned local cache of active policies; keeping recent model/provider metadata locally; using asynchronous audit persistence; using provider fallback and circuit breakers; and keeping the privacy scanner available locally or as a resilient module. The service boundary itself does not need to change.

4.3 Privacy Scanner Service

Reachable — Pass. Exposes `scanContent`, `maskSensitiveData`, and `getScanResult` operations.

Self-contained — Pass. Owns Scanner Result data only; does not read policy, identity, or audit tables.

Has a contract — Pass. `scanContent()` and related operations are explicitly defined with documented inputs, outputs, and errors.

Independent — Pass. Detection uses deterministic pattern rules against the content it is given; it does not need Identity, Policy, Model, or Audit services to run a scan.

Loosely coupled — Pass. The Gateway calls it through its contract and forwards the scan result to the Policy Service as plain data; the scanner never reaches into another service's storage.

4.4 Policy & Approval Service

Reachable — Pass. Exposes policy CRUD, evaluation and approval operations.

Self-contained — Pass. Owns Policies and Approvals.

Has a contract — Pass. Policy creation, update and evaluation operations are documented.

Independent — Pass. Policy evaluation uses its own policy data without directly querying another service.

Loosely coupled — Pass. User and request references are passed as identifiers instead of accessing Identity or Gateway tables.

4.5 Model & Provider Service

Reachable — Pass. Exposes model registration, provider management, routing and provider-health operations.

Self-contained — Pass. Owns Providers and Models; provider configuration is kept within the service boundary.

Has a contract — Pass. Operations such as `listModels`, `registerModel`, and `routeAndInvoke` are defined.

Independent — Partial. The service ultimately depends on external AI providers — if all configured providers are unavailable, an AI response cannot be produced. Retries, timeouts, circuit breakers and fallback providers reduce, but do not eliminate, this dependency.

Loosely coupled — Pass. The gateway does not know how OpenAI, Gemini or a local provider is implemented; it only uses the Model & Provider contract.

4.6 Security & Audit Service

Reachable — Pass. Exposes trace and security-event operations.

Self-contained — Pass. Owns Security Events, Request Traces, and Audit Events.

Has a contract — Pass. Operations such as `recordSecurityEvent`, `recordTrace`, and `getTrace` are defined.

Independent — Pass. Can record traces and security events without requiring the AI provider to be available.

Loosely coupled — Pass. Other services send request/trace identifiers and event information through the contract; no service directly accesses MongoDB collections owned by Security & Audit Service.

5. Summary of Fixes

Service	Property Affected	Fix (does not change the service boundary or the Task 3 contract)
AI Gateway Service	Independent (Fail)	Cache recently verified sessions for a short window; keep a versioned local cache of active policies; keep recent model/provider metadata locally; persist audit events asynchronously; rely on provider fallback and circuit breakers; keep the privacy scanner available locally or as a resilient module.
Model & Provider Service	Independent (Partial)	Configure multiple providers with automatic fallback and circuit breakers so a single provider outage does not block model selection; degrade gracefully to a cached "last known good" model list when every configured provider is down.